Closed-Loop Residual Tokenization for Stable Compression of Dynamical State Trajectories

Ivan Tyshchenko
Independent research

Abstract

Stateful model architectures produce dense trajectories whose storage cost grows linearly with both state dimension and sequence length. Directly sparsifying and quantizing consecutive state deltas appears attractive, but the decoder does not possess the dense state used by the encoder; consequently, discarded components accumulate as open-loop reconstruction error. We present VoidToken v3, a closed-loop residual representation that computes each residual against the state actually reconstructed by the decoder. Sparse top-k residual components are norm-scaled and quantized, while an automatic keyframe schedule is selected from an explicit byte budget. We evaluate the method in Core LM, a reproducible discrete-time dynamical benchmark, against dense float32 storage and PCA. The evaluation contains 115 runs spanning three state dimensions, five random seeds, five perturbation classes, short and long trajectories, three sparsity levels, and two quantization levels. Under predeclared acceptance thresholds, all 115 runs pass. The minimum observed compression ratio is 4.2353×, the worst normalized root-mean-square error is 0.06089, the minimum cosine similarity is 0.99821, and the maximum relative mean-energy drift is 0.04955. The result establishes a reproducible operating region for the tested dynamical system; it does not claim universal performance on arbitrary learned-model states or task-level language-model quality.

1 Introduction

Sequential systems often maintain a continuous state $S_t \in \mathbb{R}^n$ and update it under an external input U_t:

$$S_{t+1} = F(S_t, U_t). \tag{1}$$

Persisting the complete trajectory in float32 requires $4n(T+1)$ bytes for T transitions. This cost is material for long-running agents, recurrent models, simulation traces, online diagnostics, and replay systems. Compression is therefore useful only if it reduces actual serialized bytes while preserving the trajectory signals needed by downstream analysis.

Low-rank projection and quantization are standard tools for reducing representations [5, 2]. Recurrent systems create an additional problem: a small local error can be integrated over many steps. Error-feedback methods are known to correct bias introduced by lossy compression in iterative optimization [6, 7]. We apply the same closed-loop principle to state-trajectory storage.

This work makes four contributions:

1. We identify a decoder-state mismatch in open-loop sparse delta coding.
2. We define a closed-loop residual token with sparse indices, quantized values, and explicit reconstruction semantics.
3. We derive an automatic keyframe interval from a serialized-byte budget, rather than selecting it solely by reconstruction quality.
4. We provide a native macOS benchmark application and a reproducible 115-run evaluation with machine-readable evidence.

Figure 1 summarizes the benchmark. One materialized input stream drives one dense Core LM trajectory. Dense, PCA, and VoidToken backends then receive the identical trajectory, eliminating input variation between methods.

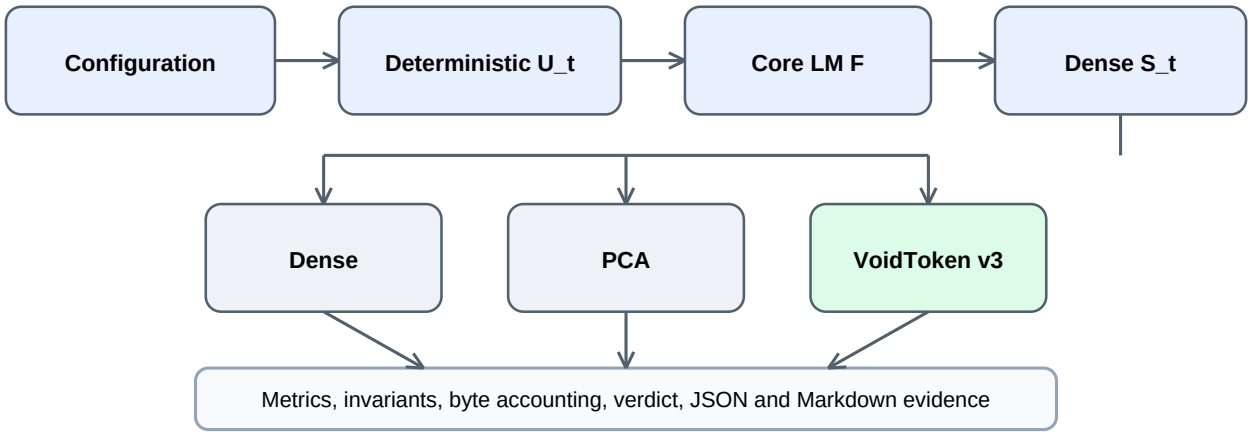

Figure 1: Benchmark data flow. Metrics and verdicts are computed outside the visualization layer from the same materialized trajectory.

2 Related Work

Low-rank representations. Principal component analysis constructs orthogonal directions of maximal variance and is a natural baseline for trajectory matrices [5]. Low-rank and tensor decompositions have also been used to compress recurrent language models [1]. PCA exploits global trajectory structure but requires a batch matrix and stores a basis, mean, and scores.

Quantization and sparse representations. Quantization maps continuous values into a finite codebook [2]. Neural-network compression uses reduced precision, pruning, and structured decomposition to reduce model storage and computation [3, 4]. Such work primarily compresses weights or activations. Our target is different: a sequential history whose decoder state changes after every lossy token.

Error feedback. One-bit stochastic-gradient methods introduced residual accumulation to recover information lost by quantization [7]. Karimireddy et al. showed that error feedback can repair biased compression operators in iterative optimization [6]. VoidToken v3 adopts the closed-loop idea for trajectory reconstruction: residuals are formed against decoder-visible state, not against inaccessible dense history.

3 Core LM Benchmark

3.1 State transition

The benchmark implements a stabilized nonlinear state transition:

$$D(S_t) = WS_t + \tanh(S_t), \tag{2}$$

$$E(S_t, H_t) = \|S_t\|_2^2 + \lambda \operatorname{Var}(H_t), \tag{3}$$

$$S_{t+1} = S_t + \alpha D(S_t) + \beta U_t - \gamma_t \nabla \|S_t\|_2^2, \tag{4}$$

where W is deterministically initialized and spectrally scaled. The stabilization coefficient γ_t increases smoothly with energy. The implementation uses $\alpha = 0.10$, $\beta = 0.20$, base $\gamma = 0.05$, and $\lambda = 0.50$. External sources can create the perturbation U_t but cannot mutate S_t directly.

3.2 Perturbation classes

Five bounded classes are used:

1. zero input;
2. clipped Gaussian input;
3. bounded uniform input;
4. a sparse impulse;
5. a repeating structured motif.

The nonzero inputs satisfy $\|U_t\|_\infty \le 0.05$. A fixed seed fully determines the input stream and transition matrix. The SHA-256 digest of the materialized input is recorded in each result.

3.3 Baselines

Dense. The dense baseline serializes all states as little-endian float32. Its payload size is measured from the byte buffer, not inferred from an object-size proxy.

PCA. For the trajectory matrix $X \in \mathbb{R}^{(T+1)\times n}$, the PCA backend stores a float32 mean, r right-singular vectors, and projection scores. It provides a global low-rank reference, but is not an online token format.

4 VoidToken v3

4.1 Failure of open-loop deltas

An open-loop encoder forms $\Delta_t = S_t - S_{t-1}$ and transmits a lossy approximation $Q(\Delta_t)$. The decoder evolves as

$$\widehat{S}_t = \widehat{S}_{t-1} + Q(S_t - S_{t-1}). \tag{5}$$

The encoder subtracts S_{t-1}, while the decoder starts from $\widehat{S}_{t-1}$. Once these differ, Eq. (5) fails to encode the existing reconstruction error. Figure 2 shows the resulting accumulation in a representative run.

4.2 Closed-loop residual

VoidToken v3 instead forms

$$r_t = S_t - \widehat{S}_{t-1}. \tag{6}$$

Let I_t contain the indices of the k largest components of r_t in absolute value and let $a_t = \|r_t\|_2$. For $i \in I_t$, the quantized value is

$$q_{t,i} = \mathrm{clip}\left(\mathrm{round}\left(q_{\max}\frac{r_{t,i}}{a_t}\right), -q_{\max}, q_{\max}\right). \tag{7}$$

The sparse decoded residual has

$$\widehat{r}_{t,i} = \begin{cases} a_t q_{t,i}/q_{\max}, & i \in I_t, \\ 0, & \text{otherwise}, \end{cases} \tag{8}$$

and the decoder updates

$$\widehat{S}_t = \widehat{S}_{t-1} + \widehat{r}_t. \tag{9}$$

The encoder performs the same update locally. Thus, every discarded component remains in the next residual. This is the critical correction.

4.3 Serialized format

A normal token contains:

- one float32 residual norm;
- one uint16 component count;
- k uint16 or uint32 indices;
- k int8 or int16 quantized values.

The first state is dense. A keyframe token uses the reserved component count 65535 followed by one dense float32 state.

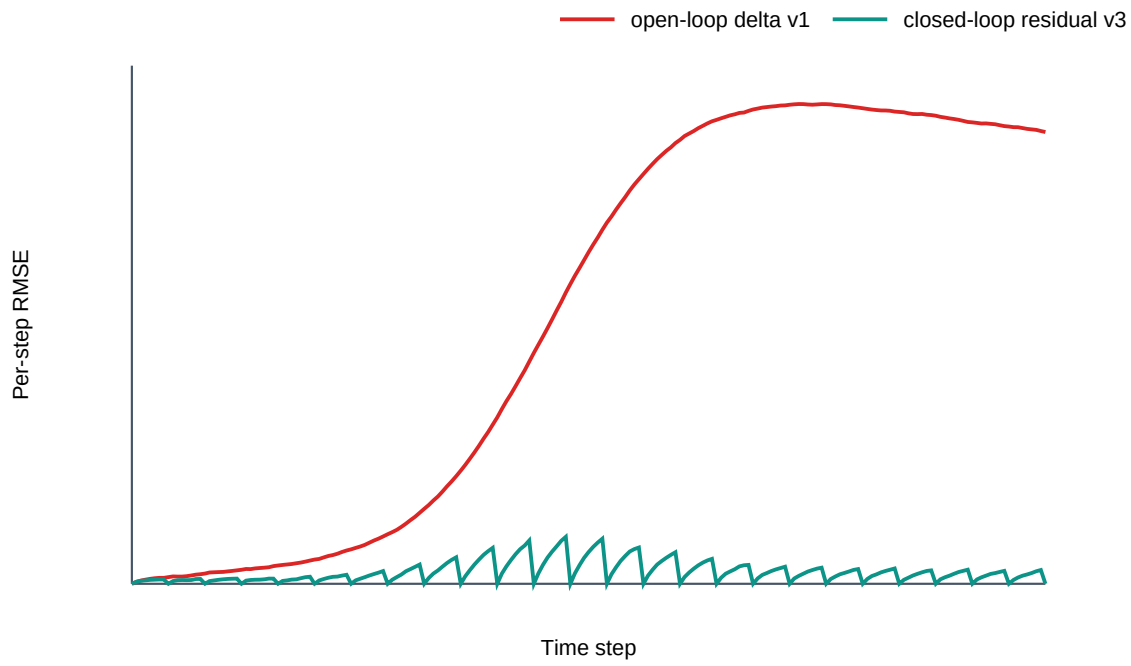

Figure 2: Per-step RMSE for open-loop delta v1 and closed-loop residual v3 on $n = 96$, Gaussian bounded input, seed 42.

4.4 Byte-budgeted keyframes

Let

$$B_D = 4n, \tag{10}$$
$$B_T = 6 + k(b_{\text{idx}} + b_q), \tag{11}$$
$$B_K = 6 + 4n \tag{12}$$

denote dense-state, normal-token, and keyframe-token bytes. For an internal safety target $\rho_s = 4.1$, the mean byte budget is B_D/ρ_s. The smallest keyframe interval compatible with this target satisfies

$$L \geq \frac{B_K - B_T}{B_D/\rho_s - B_T}. \tag{13}$$

The implementation rounds L upward to a power of two and enforces $L \geq 8$. If no feasible denominator exists, automatic keyframes are disabled and the ordinary verdict detects a compression failure. Zero residuals do not emit unnecessary dense keyframes.

5 Evaluation

5.1 Protocol

The full matrix contains 115 unique runs:

- dimensions $n \in \{32, 96, 256\}$;
- seeds $\{7, 17, 42, 101, 997\}$;
- all five perturbation classes;
- $T \in \{200, 5000\}$;

Table 1: Worst-case VoidToken v3 results by perturbation class. Ratio is reported as dense payload bytes divided by compressed payload bytes.

Scenario	Runs	Min. ratio	Max. NRMSE	Min. cosine	Max. energy drift
zero	18	19.3735	0.00000	1.00000	0.00000
gaussian bounded	43	4.2353	0.06089	0.99821	0.02545
uniform bounded	18	5.0172	0.05191	0.99870	0.02257
impulse	18	5.0172	0.05647	0.99865	0.04955
repeating structured	18	5.0172	0.04606	0.99898	0.02085
All	115	4.2353	0.06089	0.99821	0.04955

- $k \in \{4, 8, 16\}$;
- $q_{\max} \in \{127, 32767\}$.

Canonical sparsity is $k = 4, 8, 16$ for $n = 32, 96, 256$, respectively. The additional sweep varies k and $q_{\max}$ at $n = 96$. Experiments were run on Apple silicon under macOS 26.3 with Python 3.12.13 and NumPy 2.3.5. Every run records the complete configuration, environment, input digest, method metrics, time series, invariant checks, and verdict.

5.2 Metrics and acceptance criteria

For reference trajectory X and reconstruction $\widehat{X}$, we measure

$$\mathrm{RMSE} = \sqrt{\frac{1}{(T+1)n}\|X - \widehat{X}\|_F^2}, \tag{14}$$

$$\mathrm{NRMSE} = \frac{\mathrm{RMSE}}{\sqrt{\frac{1}{(T+1)n}\|X\|_F^2}}, \tag{15}$$

$$\mathrm{cosine} = \frac{\langle X, \widehat{X}\rangle}{\|X\|_F \|\widehat{X}\|_F}. \tag{16}$$

We also measure maximum absolute error, mean-energy drift, CSI drift, energy-drift difference, payload bytes, file bytes, encode/decode time, throughput, peak traced memory, invariant violations, and deterministic replay.

A run passes only if VoidToken simultaneously achieves:

$$\rho \geq 4, \quad \mathrm{NRMSE} \leq 0.10, \quad \mathrm{cosine} \geq 0.95, \quad \Delta E_{\mathrm{rel}} \leq 0.05, \tag{17}$$

with zero invariant violations and exact deterministic replay.

5.3 Results

All 115 runs pass every criterion. Across the complete matrix, the minimum compression ratio is 4.2353×, the maximum NRMSE is 0.06089, the minimum cosine similarity is 0.99821, and the maximum relative mean-energy drift is 0.04955. The impulse class is closest to the energy-drift boundary, whereas the sparsity and precision sweep produces the minimum compression ratio.

5.4 Reproducibility and safeguards

The benchmark includes several safeguards against selective reporting:

1. the same materialized U_t is used for all storage methods;
2. failed configurations remain valid artifacts rather than being silently dropped;
3. actual binary buffers determine payload size;
4. all five declared seeds are included;
5. the aggregate references exact run identifiers;

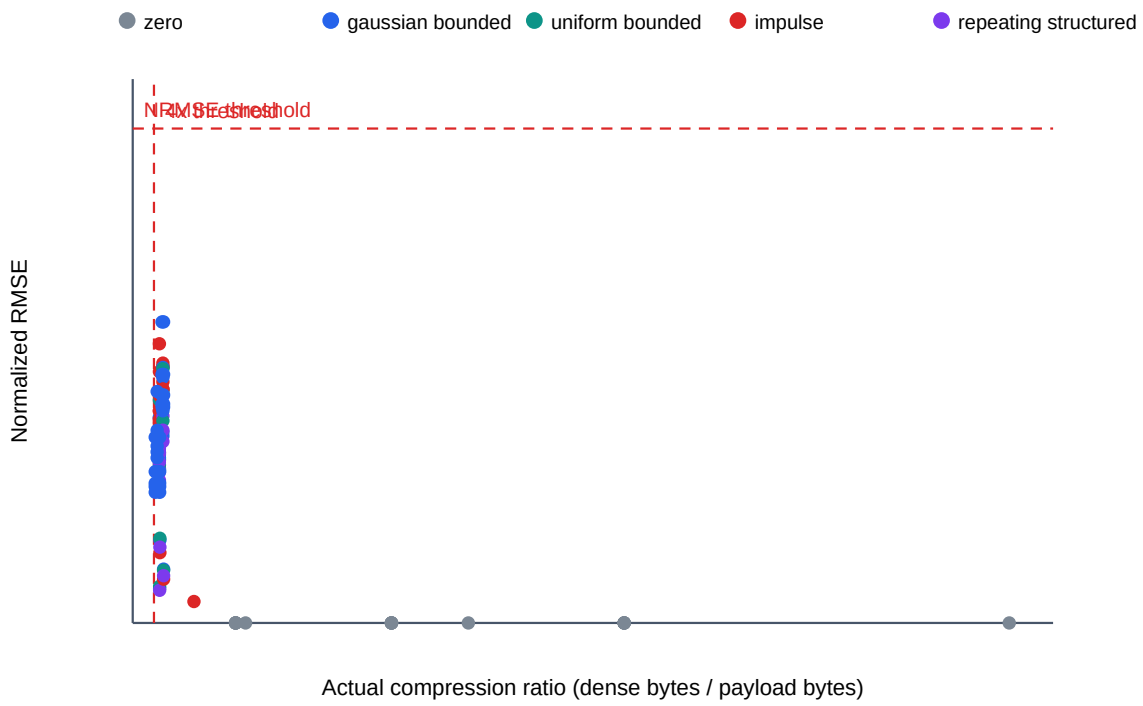

Figure 3: All 115 VoidToken v3 runs in compression-NRMSE space. Every point lies to the right of the 4× boundary and below NRMSE 0.10.

6. a replay must reproduce inputs and states exactly;
7. the UI reads generated JSON and contains no synthetic chart data.

The accompanying artifact contains the benchmark core, suite runner, tests, aggregate, and a native SwiftUI inspection application.

6 Discussion

Why v3 succeeds. Top-k sparsification alone is not sufficient for a sequential decoder. The essential change is semantic: the residual is defined relative to the decoder's own state. This closes the loop. Keyframes then bound transient error without violating the byte target because their interval is derived from serialized costs.

Compression-quality tradeoff. Increasing k reduces residual error but enlarges each token. Increasing $q_{\max}$ reduces scalar quantization error but may double value width. Equation (13) couples these choices with keyframe frequency. The observed minimum ratio of 4.2353× demonstrates that the internal 4.1× safety target leaves a measurable margin above the public 4× criterion.

PCA comparison. PCA can provide strong reconstruction when trajectory rank is low, but its scores and basis may fail the 4× byte target for some shapes, and it requires batch access to the trajectory. VoidToken is sequential and directly decodable token by token. The two representations therefore address different deployment constraints.

7 Limitations

The evaluation is deliberately bounded. First, inputs are deterministic synthetic perturbations rather than telemetry collected from deployed language models. Second, dimensions are limited to 32, 96, and 256. Third, energy, CSI,

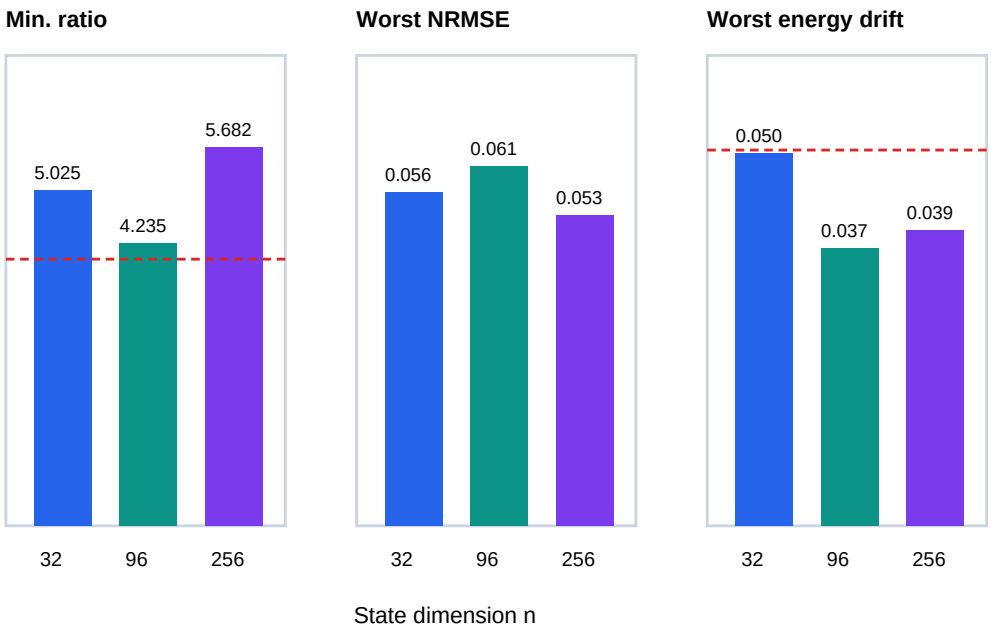

Figure 4: Worst-case metrics grouped by state dimension. Red guide lines mark applicable acceptance boundaries.

and state reconstruction are system-level proxies; they do not measure task-level language quality. Fourth, the benchmark evaluates one nonlinear transition family and does not establish a theorem for arbitrary F. Fifth, Python traced memory excludes all native allocations made by NumPy. Finally, the automatic keyframe rule guarantees only a nominal byte budget under the specified token layout; the measured verdict remains necessary.

8 Conclusion

Open-loop delta quantization can look locally accurate while accumulating global state error. VoidToken v3 corrects the mismatch by encoding residuals against decoder-visible state and scheduling keyframes from explicit byte accounting. In a 115-run Core LM benchmark, the method satisfies all predeclared compression, reconstruction, energy, invariance, and determinism criteria. The result supports further evaluation on real model telemetry while providing a reproducible baseline and a concrete failure-correction narrative.

Artifact Availability

Source code, tests, the registered 115-run evidence set, paper sources, and reproduction instructions are publicly available at `https://github.com/ALLPROTO/core-lm-benchmark`. The software is released under the MIT License.

References

[1] Artem M. Grachev, Dmitry I. Ignatov, and Andrey V. Savchenko. Compression of recurrent neural networks for efficient language modeling. *Applied Soft Computing*, 79:354–362, 2019.

[2] Robert M. Gray and David L. Neuhoff. Quantization. *IEEE Transactions on Information Theory*, 44(6):2325–2383, 1998.

[3] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.

[4] Qinyao He, He Wen, Shuchang Zhou, Yuxin Wu, Cong Yao, Xinyu Zhou, and Yuheng Zou. Effective quantization methods for recurrent neural networks. *CoRR*, abs/1611.10176, 2016.

[5] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933.

[6] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error feedback fixes signsgd and other gradient compression schemes. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3252–3261, 2019.

[7] Frank Seide, Hao Fu, Brian Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and application to data-parallel distributed training of speech dnns. In *Proceedings of Interspeech*, pages 1058–1062, 2014.

A Exact benchmark gate

The verdict is the conjunction of six checks: compression ratio, NRMSE, cosine similarity, mean-energy drift, invariant violations, and deterministic replay. A result is marked `INCONCLUSIVE` only when a required representation or measurement is absent. Otherwise any failed check produces `FAIL`; no partial-pass label exists.

B Artifact contents

The reproducibility archive contains:

- the Core LM transition and compression implementation;
- the 115-run suite generator;
- unit, regression, invariant, and report tests;
- the authoritative aggregate and referenced run JSON files;
- the paper figure generator;
- commands for rebuilding the macOS application and rerunning experiments.